

LifeTrack Frontend

Complete Interview Question Bank

200 Questions · 15 Topics · Angular v21 · TypeScript · RxJS

Every Angular frontend topic behind the LifeTrack clinical trial platform

Angular Architecture · Components & Lifecycle · Modules & Lazy Loading

Routing & Guards · Forms (Template & Reactive) · HTTP & Interceptors

RxJS & Observables · Services & DI · State Management · Pipes

TypeScript · Security · Performance · Project-Specific Deep Dives

Table of Contents | # | Topic | Questions | |---|---|---| | 1 | Angular Architecture & Core Concepts | Q1 - Q15 | | 2 | Components — Lifecycle, Inputs, Outputs | Q16 - Q35 | | 3 | Modules, Lazy Loading & AppModule | Q36 - Q50 | | 4 | Routing, Guards & Navigation | Q51 - Q70 | | 5 | Template-Driven vs Reactive Forms | Q71 - Q95 | | 6 | HTTP Client & Interceptors | Q96 - Q115 | | 7 | RxJS & Observables | Q116 - Q135 | | 8 | Services, Dependency Injection & State | Q136 - Q150 | | 9 | Pipes, Directives & Templates | Q151 - Q165 | | 10 | TypeScript in Angular | Q166 - Q175 | | 11 | Security Patterns in the Frontend | Q176 - Q183 | | 12 | Performance & Build | Q184 - Q190 | | 13 | Project-Specific Deep Dives | Q191 - Q200 |

Topic 1: Angular Architecture & Core Concepts

Q1.

What is Angular? What are its key architectural building blocks? **A:** Angular is a TypeScript-based open-source web application framework by Google for building single-page applications (SPAs). Key building blocks: **Modules** (`NgModule`) — organize the app into cohesive blocks. **Components** — the UI building blocks (`@Component` with template + styles). **Services** — business logic and data, injected via DI. **Directives** — extend HTML behavior (`*ngIf`, `*ngFor`). **Pipes** — transform values in templates. **Routing** — SPA navigation without page reloads. **Dependency Injection** — services provided and injected by the framework. Your LifeTrack project uses all of these across `AppModule`, `DashboardModule`, `AuthModule`, `SharedModule`, `AuthService`, `AuthGuard`, `RoleGuard`, `AuthInterceptor`, and the `ist-date` pipe.

Q2.

What is the difference between Angular and AngularJS? **A:** AngularJS (1.x) — JavaScript, two-way data binding via dirty-checking `$scope`, controller-based, no CLI, not suitable for large apps. Angular (2+) — TypeScript, component-based architecture, unidirectional data flow by default, Angular CLI, zone.js for change detection, proper DI with hierarchical injectors. Your project uses Angular v21 (confirmed by `standalone: false` component pattern and `@angular-devkit/build-angular`).

Q3.

What is a Single Page Application (SPA)? How does Angular implement it? **A:** An SPA loads a single HTML page and dynamically updates content as users interact, without full page reloads. Angular implements it via the `RouterModule` — navigation changes the URL (e.g., `/dashboard/admin`) but Angular intercepts the navigation, loads the matching component into the `<router-outlet>`, and updates the `<title>` tag, all without a server request. Your `app.component.html` has one `<router-outlet>` that hosts the entire app. `AppComponent.ngOnInit()` listens to `NavigationEnd` events to update the page title dynamically.

Q4.

What is `zone.js` and why does Angular need it? **A:** `zone.js` patches browser async APIs (`setTimeout`, `Promise`, `XHR`, `addEventListener`) to notify Angular when async operations complete. Angular's default change detection (Zone-based) triggers after every zone-aware async operation to re-render the view. Without `zone.js`, Angular wouldn't know when to re-render after an HTTP response or a `setTimeout` completes. Your `angular.json` includes `"polyfills": ["zone.js"]`.

Q5.

What is the difference between `standalone: true` and `standalone: false` components? **A:** `standalone: true` — the component is self-contained; it imports its own dependencies directly (no `NgModule` required). Angular 17+ default. `standalone: false` — the component must be declared in an `NgModule`'s `declarations` array. Your entire project uses `standalone: false` — configured globally in `angular.json` via `"@schematics/angular:component": { "standalone": false }`. This means every component must be declared in `DashboardModule`, `AuthModule`, or `AppModule`.

Q6.

What is the Angular CLI? What commands did you use for this project? **A:** The Angular CLI (`ng`) is a command-line tool for scaffolding, building, and serving Angular projects. Key commands: `ng new` — creates a new project. `ng serve` — starts the dev server (Webpack bundler + live reload). `ng generate component` — creates a new component. `ng build` — compiles to `dist/`. `ng build --configuration=production` — optimized production build with file replacement (uses `environment.prod.ts`). Your `angular.json` configures the dev server to run on port 62536 (`"port": 62536` under `serve.options`).

Q7.

What is the `environment.ts` file and how does the production build use it? **A:** `src/environments/environment.ts` — development config;

```
export const environment = { production: false, apiUrl: 'http://localhost:5046/gateway' };
```

`src/environments/environment.prod.ts` — production config (different `apiUrl`). In `angular.json` under `configurations.production`:

```
"fileReplacements": [{ "replace": "src/environments/environment.ts", "with": "src/environments/environment.prod.ts" }]
```

At build time, Angular replaces the dev file with the prod file. All your services and components import from `../environments/environment` — they automatically get the right `apiUrl` for each environment.

Q8.

What is the difference between `AppModule`, `DashboardModule`, `AuthModule`, and `SharedModule` in your project? **A:** `AppModule` — root module, bootstraps `AppComponent`, registers `HttpClientModule`, `AuthInterceptor`, and `DATE_PIPE_DEFAULT_OPTIONS`. `AuthModule` — lazy-loaded feature module for login and register pages. `DashboardModule` — lazy-loaded feature module containing all

28+ role-based pages and routes. `SharedModule` — re-usable components module (`PageHeaderComponent` , `EmptyStateComponent`) imported by `DashboardModule` . Feature modules keep the initial bundle small via lazy loading.

Q9.

What is `@NgModule` ? What are its key properties? **A:** `@NgModule` is a class decorator that defines an Angular module. Key properties: `declarations` — components, directives, pipes owned by this module. `imports` — other modules this module depends on (e.g., `CommonModule` , `RouterModule` , `ReactiveFormsModule`). `providers` — services (though most use `providedIn: 'root'`). `exports` — declarations other modules can use after importing this module. `bootstrap` — root component(s) to instantiate when app starts (only in `AppModule`). Your `DashboardModule` has 30+ components in `declarations` and imports `FormsModule` , `ReactiveFormsModule` , `SharedModule` , `RouterModule.forChild(routes)` .

Q10.

What is `platformBrowserDynamic` and `bootstrapModule` ? **A:** `platformBrowserDynamic().bootstrapModule(AppModule)` in `main.ts` is the application entry point. It loads the Angular framework in the browser (JIT in development, AOT in production) and bootstraps `AppModule` , which in turn renders `AppComponent` (`<app-root>`) into `index.html` 's `<app-root>` element.

Q11.

What is the difference between JIT and AOT compilation? **A: JIT (Just-in-Time)** — compiles templates in the browser at runtime. Slower startup, larger bundle (includes compiler), easier debugging. Used in `ng serve` development. **AOT (Ahead-of-Time)** — compiles templates at build time. Smaller bundle (no compiler shipped), faster startup, catches template errors at build time. Used in `ng build --configuration=production` . Your `angular.json` sets `"optimization": true` for production and `"optimization": false` for development.

Q12.

What is the `selector` in a component? What does `app-` prefix mean? **A:** `selector: 'app-login'` is a CSS selector that tells Angular where to render this component — wherever `<app-login></app-login>` appears in a template. The `app-` prefix is a convention to avoid conflicts with native HTML elements and third-party libraries. Configured globally in `angular.json` as `"prefix": "app"` . Custom element selectors should always have a prefix.

Q13.

What is the `templateUrl` vs inline `template` ? What does your project use? **A:** `templateUrl: './login.component.html'` — references an external HTML file, better for IDE support and complex templates. `template: '<p>Hello</p>'` — inline string, good for tiny templates. Your project uses `templateUrl` for every component with a real template. Exception: `RoleHomeComponent` uses inline `template: ''` — an empty template, because it's a pure navigation component (redirects immediately in `ngOnInit` , never renders anything).

Q14.

What is change detection? What are the two change detection strategies? **A:** Change detection is Angular's mechanism to synchronize the model (component data) with the view (DOM). Angular walks the component tree and re-renders any component whose bound data has changed. **Default** — checks every component on every change detection cycle (triggered by `zone.js` after any async event). **OnPush** — only checks when `@Input()` reference changes, an event emitter fires, or `markForCheck()` is called. Most efficient. Your project uses Default strategy (not explicitly set). For a large app like LifeTrack, `OnPush` on list components would improve performance.

Q15.

What is `ViewEncapsulation` ? What modes exist? **A:** `ViewEncapsulation` controls how component styles are scoped. **Emulated** (default) — Angular adds unique attributes (`_ngghost-xxx`) to scope styles to the component. Your `.css` files' styles don't leak to other components. **None** — global styles, no encapsulation. **ShadowDom** — uses native browser Shadow DOM for true encapsulation. Your project uses the default `Emulated` since you specify `styleUrls: ['./login.component.css']` without overriding encapsulation.

Topic 2: Components — Lifecycle, Inputs, Outputs

Q16.

What are Angular lifecycle hooks? List all 8 in order. **A:** In order: `ngOnChanges` → `ngOnInit` → `ngDoCheck` → `ngAfterContentInit` → `ngAfterContentChecked` → `ngAfterViewInit` → `ngAfterViewChecked` → `ngOnDestroy`. The most commonly used in your project are `ngOnInit` (fetch data, set up subscriptions) and `ngOnDestroy` (clean up Subject subscriptions).

Q17.

What is `ngOnInit`? Why use it instead of the constructor? **A:** `ngOnInit` runs after Angular has set all `@Input()` properties and initialized the component. The constructor runs before inputs are set — data from the parent isn't available yet. Rule: **constructor for DI only**, `ngOnInit` for initialization logic. In `AdminDashboardComponent.ngOnInit()`: calls `forkJoin({...})` to load dashboard stats. In `NotificationsPageComponent.ngOnInit()`: fires the HTTP GET for notifications. In `LoginComponent` constructor: only injects services and builds the form (safe). `if (this.auth.isLoggedIn()) this.router.navigate(...)` also in constructor — acceptable because `AuthService` doesn't depend on `@Input()`.

Q18.

What is `ngOnDestroy`? Why is it critical for subscription management? **A:** `ngOnDestroy` runs just before Angular destroys the component. Use it to: unsubscribe from Observables, clear timers (`clearTimeout`), close WebSocket connections. In `AdminDashboardComponent`:

```
private destroy$ = new Subject<void>();

ngOnInit() {
  forkJoin({...}).pipe(takeUntil(this.destroy$)).subscribe(...);
}

ngOnDestroy(): void {
  this.destroy$.next();
  this.destroy$.complete();
}
```

Without `takeUntil(this.destroy$)`, the subscription would leak — it would stay alive even after the component is destroyed (e.g., user navigates away), causing memory leaks and executing callbacks on dead components. `AdminSitesPageComponent` and `AdminProtocolsPageComponent` also call `clearTimeout(this.searchTimer)` in `ngOnDestroy` to cancel pending debounce timers.

Q19.

What is `ngOnChanges`? When does it fire? **A:** Fires whenever a bound `@Input()` property value changes — before `ngOnInit` on the first change, then after every subsequent change. Receives a `SimpleChanges` object with `previousValue` and `currentValue`. Used for reacting to parent data changes. `PageHeaderComponent` has `@Input() title!: string` and `@Input() subtitle?: string` — if the parent changes `[title]`, `ngOnChanges` would fire. Your project doesn't explicitly implement `ngOnChanges` anywhere, but understanding it is interview-critical.

Q20.

What is `@Input()` and `@Output()`? Explain with your `PageHeaderComponent` example. **A:** `@Input()` — data flows **into** the child component from the parent. `@Output()` — events flow **out** of the child to the parent. In `PageHeaderComponent`:

```
@Input() title!: string; // parent sets title="Documents"
@Input() subtitle?: string; // optional subtitle
@Output() back = new EventEmitter<void>(); // child emits back event

onBack(): void { this.back.emit(); }
```

Parent usage: `<app-page-header title="Documents" (back)="goBack()">`. When user clicks the back button, `back.emit()` fires, the parent's `goBack()` method is called. `EventEmitter<void>` means no data is passed with the event.

Q21.

What is `EventEmitter`? How is it different from an RxJS `Subject`? **A:** `EventEmitter` extends RxJS `Subject` — it's a `Subject` with an Angular-specific `emit()` method (alias for `next()`). Use `EventEmitter` for `@Output()` in components. Use `Subject` for internal app state and RxJS pipelines. `EventEmitter<void>` emits with no payload. `EventEmitter<string>` would emit a string value. In `PageHeaderComponent`, `@Output() back = new EventEmitter<void>()` — void because you just need to know the back button was clicked, not what value was passed.

Q22.

What is the `!` (non-null assertion) operator in TypeScript? Where is it used in your project? **A:** `!` tells TypeScript "I know this is not null/undefined — don't warn me." In `PageHeaderComponent`: `@Input() title!: string` — the `!` says "Angular will definitely provide this before use." In form accessor methods: `this.form.get('email')!` — EF Core method returns

`AbstractControl | null`, the `!` asserts it won't be null. Use carefully — if the value IS null at runtime, you get a runtime error instead of a compile-time warning.

Q23.

What is `ng-content` and content projection? How is it used in `PageHeaderComponent`? **A:** `ng-content` allows a parent to inject HTML into a child component's template — content projection. In `PageHeaderComponent`'s template: `<ng-content></ng-content>` renders whatever HTML the parent puts between the tags. Usage: `<app-page-header title="Documents" (back)="goBack()"><button (click)="openModal()"> New</button></app-page-header>`. The `<button>` is projected into the header's right slot. This lets `PageHeaderComponent` be reused across all pages while each page can add its own action button.

Q24.

What is `ViewChild`? When would you use it instead of `@Input()`? **A:** `@ViewChild(ComponentOrRef)` gives the parent direct access to a child component instance or DOM element. Use it when you need to call a child's method or access DOM properties imperatively. `@Input()` is for data binding (declarative). Example: if a modal component had a `reset()` method, you'd use `@ViewChild(ModalComponent) modal!: ModalComponent` then call `this.modal.reset()`. Your project manages modals with boolean flags (`showCreateModal = true`) — purely declarative, no `ViewChild` needed.

Q25.

What is the `constructor` used for in Angular components? **A:** Only for dependency injection. Best practice: inject services in the constructor, do all other initialization in `ngOnInit`. Why? (1) `ngOnInit` runs after inputs are set. (2) Constructor runs during component instantiation — heavy work here blocks rendering. (3) Better testability — easier to mock services in constructor injection. In `LoginComponent`, the constructor injects `FormBuilder`, `AuthService`, `Router`, builds the form, and checks if already logged in — acceptable because none of these depend on `@Input()`.

Q26.

What are smart components vs dumb components? Classify your project's components. **A: Smart (container)** — knows about services, makes HTTP calls, manages state. Dashboard components (`AdminDashboardComponent`, `AdminProtocolsPageComponent`) are smart — they inject `HttpClient`, `DashboardService`, call APIs, manage `loading`, `error`, pagination state. **Dumb (presentational)** — receives data via `@Input()`, emits events via `@Output()`, no service injection. `PageHeaderComponent` and `EmptyStateComponent` are dumb — pure presentation. This separation makes dumb components reusable and testable without mocking services.

Q27.

How does `standalone: false` affect where you can use a component? **A:** A `standalone: false` component must be declared in exactly one `NgModule`'s `declarations` array. It can only be used in templates within that module (or modules that import it). You cannot use `AdminDashboardComponent` in `AuthModule` without cross-module declarations. All 30+ dashboard components are declared in `DashboardModule`. `AppComponent` is declared in `AppModule`. `LoginComponent` and `RegisterComponent` are declared in `AuthModule`.

Topic 3: Modules, Lazy Loading & AppModule

Q28.

What is lazy loading in Angular? Why is it critical for LifeTrack? **A:** Lazy loading means feature modules are not loaded with the initial bundle — they're downloaded on demand when the user navigates to their routes. In `AppRoutingModule`:

```
{ path: 'auth', loadChildren: () => import('./features/auth/auth.module').then(m => m.AuthModule) },
{ path: 'dashboard', loadChildren: () => import('./features/dashboard/dashboard.module').then(m => m.DashboardModule),
  canActivate: [AuthGuard] }
```

Without lazy loading, all 30+ role-specific components would be downloaded on the first page load — even if the user never visits the admin pages. With lazy loading: initial bundle contains only `AppModule` + core services. `AuthModule` downloads when user hits `/auth`. `DashboardModule` downloads when user logs in. Dramatically reduces Time-to-Interactive.

Q29.

What is `RouterModule.forRoot()` vs `RouterModule.forChild()`? **A:** `forRoot(routes)` — used **once** in `AppRoutingModule` (the root module). Registers the Router service and root-level routes. Calling `forRoot` twice would create two Router instances and break navigation. `forChild(routes)` — used in **feature modules** (`AuthModule`, `DashboardModule`). Adds child routes to the existing router without creating a new Router instance. Your `AuthModule` and `DashboardModule` both use `RouterModule.forChild(routes)`.

Q30.

What is `CommonModule` and why do feature modules import it? **A:** `CommonModule` provides Angular's common directives and pipes: `*ngIf`, `*ngFor`, `*ngSwitch`, `NgClass`, `NgStyle`, `AsyncPipe`, `DatePipe`, `CurrencyPipe`, etc. `BrowserModule` (in `AppModule`) already includes `CommonModule`, but feature modules must import `CommonModule` directly since they don't extend `AppModule`. Your `AuthModule`, `DashboardModule` all import `CommonModule`. `SharedModule` re-exports `CommonModule` so `DashboardModule` gets it through the `SharedModule` import.

Q31.

What is the difference between `FormsModule` and `ReactiveFormsModule`? **A:** `FormsModule` — enables template-driven forms (`ngModel`, `ngForm`). Needed for `[(ngModel)]` two-way binding. `ReactiveFormsModule` — enables reactive forms (`FormBuilder`, `FormGroup`, `FormControl`, `Validators`). Needed for `this.fb.group({...})` and `[formGroup]`. Your project uses **both**. `AuthModule` imports only `ReactiveFormsModule` (login/register use `FormBuilder`). `DashboardModule` imports both — some pages use reactive forms (create/edit modals), some pages use simple `[(ngModel)]` for inline search/filter inputs.

Q32.

Why does `AppModule` provide `AuthInterceptor` and `DATE_PIPE_DEFAULT_OPTIONS`? **A:** `AppModule.providers` is the root DI container — anything provided here is available application-wide. `AuthInterceptor` needs to intercept ALL HTTP requests, so it must be registered at root:

```
{ provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }
```

`DATE_PIPE_DEFAULT_OPTIONS` sets the default timezone for Angular's built-in `DatePipe` globally:

```
{ provide: DATE_PIPE_DEFAULT_OPTIONS, useValue: { timezone: '+0530' } }
```

IST (UTC+5:30) — so every `{{ date | date }}` in the entire app formats in Indian Standard Time.

Q33.

What is `multi: true` in a provider? Why does `AuthInterceptor` need it? **A:** `multi: true` tells Angular that multiple values can be registered under the same injection token — it collects them into an array. `HTTP_INTERCEPTORS` is an `InjectionToken<HttpInterceptor[]>`. Multiple interceptors can be registered. If you had Auth + Logging interceptors:

```
{ provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true },
{ provide: HTTP_INTERCEPTORS, useClass: LoggingInterceptor, multi: true }
```

Without `multi: true`, the second registration would overwrite the first. With it, Angular injects an array `[AuthInterceptor, LoggingInterceptor]` and runs both in order.

Q34.

What is the `SharedModule` pattern? Why not just import `CommonModule` directly everywhere? **A:** `SharedModule` is a module that re-exports commonly needed modules AND declares reusable components. `DashboardModule` imports `SharedModule` and gets: `CommonModule`, `PageHeaderComponent`, `EmptyStateComponent` — in one import. Without `SharedModule`, every feature module would need: `imports: [CommonModule, ...]` AND separately declare shared components. With `SharedModule`: consistent set of imports

everywhere, one place to add new shared components. Your `SharedModule` exports `CommonModule`, `PageHeaderComponent`, `EmptyStateComponent`.

Q35.

What is a chunk in Angular's build output? How does lazy loading affect it? **A:** When Angular builds, it creates JavaScript bundles. Without lazy loading: one large `main.js`. With lazy loading: `main.js` (core app) + separate chunks per lazy module: `auth-module.js`, `dashboard-module.js`. Angular names them based on the dynamic import path. These chunks are only downloaded when the user first navigates to that route. Your `angular.json` production config uses `"outputHashing": "all"` — appends a hash to filenames (`dashboard-module.abc123.js`) for cache-busting.

Topic 4: Routing, Guards & Navigation

Q36.

What is Angular Router and how does it work without page reloads? **A:** Angular Router intercepts URL changes (via the HTML5 History API — `pushState`) and renders the matching component into `<router-outlet>` without a full browser navigation. When user clicks a `[routerLink]` or calls `router.navigate(...)`, Angular: (1) matches the new URL to a route definition, (2) runs any guards, (3) lazy-loads the module if needed, (4) instantiates the component, (5) renders it in `<router-outlet>`, (6) updates the browser URL. The server never receives the navigation request.

Q37.

Explain the route configuration in your `AppRoutingModule`. What does each property mean? **A:**

```
const routes: Routes = [
  { path: 'auth', loadChildren: () => import('./features/auth/auth.module').then(m => m.AuthModule) },
  { path: 'dashboard', loadChildren: () => import('./features/dashboard/dashboard.module').then(m => m.DashboardModule),
  canActivate: [AuthGuard] },
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
  { path: '**', redirectTo: 'dashboard' }
];
```

`path: ''` with `pathMatch: 'full'` — only matches the exact empty URL. `path: '**'` — wildcard, matches any unmatched URL (404 handler). Both redirect to dashboard. `canActivate: [AuthGuard]` — runs `AuthGuard.canActivate()` before loading `DashboardModule`. If it returns `false`, navigation is blocked.

Q38.

What is `canActivate`? Explain your `AuthGuard`. **A:** `canActivate` is a route guard that runs before navigation to a route. Returns `boolean` (or `Observable<boolean>`, `Promise<boolean>`). If `true`, navigation proceeds. If `false`, navigation is blocked.

```
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  canActivate(): boolean {
    if (this.auth.isLoggedIn()) return true;
    this.router.navigate(['/auth/login']);
    return false;
  }
}
```

Applied to `dashboard` in `AppRoutingModule`. If `isLoggedIn()` is false (no token or expired), redirects to login and blocks navigation. Prevents unauthenticated users from seeing the dashboard.

Q39.

What is your `RoleGuard`? How does it use `route.data`? **A:** `RoleGuard` is a fine-grained authorization guard applied to individual pages. It reads the allowed roles from `route.data` and compares to the user's JWT role:

```
canActivate(route: ActivatedRouteSnapshot): boolean {
  if (!this.auth.isLoggedIn()) { this.router.navigate(['/auth/login']); return false; }
  const allowed: string[] = route.data?.['roles'] ?? [];
  if (allowed.length === 0) return true; // no restriction
  const tokenRole = this.auth.getRoleFromToken();
  const userRole = tokenRole ?? this.auth.currentUser?.role ?? '';
  if (allowed.includes(userRole)) return true;
  this.router.navigate(['/dashboard']); // wrong role → redirect
  return false;
}
```

Route definition: `{ path: 'admin/users', component: AdminUsersPageComponent, canActivate: [RoleGuard], data: { roles: ['Admin'], title: 'User Management' } }`. Both `AuthGuard` (is logged in?) and `RoleGuard` (right role?) can be combined on the same route.

Q40.

What is `ActivatedRoute` and `ActivatedRouteSnapshot`? How does `RoleGuard` use it? **A:** `ActivatedRoute` — the currently active route, live observable-based. `ActivatedRouteSnapshot` — a static snapshot of the route at a point in time (immutable). `RoleGuard.canActivate(route: ActivatedRouteSnapshot)` receives the snapshot of the route being navigated to, allowing it to read `route.data?.['roles']` — the static data declared in the route config.

Q41.

What is the difference between `router.navigate()` and `router.navigateByUrl()`? **A:** `navigate(['/dashboard/admin'])` — takes an array of path segments; relative and absolute navigation supported; can pass query params as second argument: `navigate(['/dashboard'], { queryParams: { section: 'reports' } })`. `navigateByUrl('/dashboard/admin')` — takes a full URL string; always absolute. Your project uses `navigate(...)` everywhere for type-safe segment arrays.

Q42.

What is `replaceUrl: true` in navigation? Where did you use it? **A:** `replaceUrl: true` replaces the current history entry instead of pushing a new one. Without it, pressing the browser back button from `/dashboard/admin` would go back to `/dashboard` (the empty path that redirects). With `replaceUrl: true`, the redirect doesn't add a history entry:

```
this.router.navigate([target], { replaceUrl: true }); // in RoleHomeComponent
```

So the browser back button from `/dashboard/admin` goes back to whatever the user was on before they logged in, not back to `/dashboard` (which would immediately redirect again).

Q43.

What is the `<router-outlet>` directive? What is a nested router outlet? **A:** `<router-outlet>` is the placeholder where the Router renders the matched component. Your `app.component.html` has one root `<router-outlet>`. `DashboardComponent.html` has a nested `<router-outlet>` for the child routes. The route config:

```
{ path: '', component: DashboardComponent, children: [
  { path: 'admin', component: AdminDashboardComponent },
  { path: 'admin/users', component: AdminUsersPageComponent, canActivate: [RoleGuard], ... }
]}
```

`DashboardComponent` is always rendered (it contains the sidebar/header), and the nested `<router-outlet>` inside it renders the active child page.

Q44.

What is `routerLink` vs `router.navigate()`? Which does your project use? **A:** `[routerLink]='[/dashboard/admin']'` — declarative, used in templates. Angular handles `href` generation and click. `this.router.navigate(['/dashboard/admin'])` — imperative, used in component logic. Your project uses both: `router.navigate()` in `RoleHomeComponent` (programmatic redirect), `DashboardComponent` logout handler, and page navigation methods (`manageUsers()`, `manageProtocols()`). Buttons with click handlers use `router.navigate()`.

Q45.

What is `route.snapshot.data` vs `route.data`? Which did `AppComponent` use for page titles? **A:** `route.snapshot.data` — static snapshot, read once. `route.data` — Observable, reacts to future changes. `AppComponent.ngOnInit()` subscribes to `router.events.pipe(filter(e => e instanceof NavigationEnd))`, then walks down to the deepest active child route: `let route = this.activatedRoute; while (route.firstChild) route = route.firstChild;` and reads `route.snapshot.data?.['title']` — static read after navigation completes. Sets the page title: "Documents – Regulatory Officer | LifeTrack".

Q46.

What is the `NavigationService` in your project and why was it needed? **A:** `NavigationService` tracks the previous URL for "go back" navigation:

```
this.router.events.pipe(filter(e => e instanceof NavigationEnd)).subscribe(e => {
  this._previousUrl = this._currentUrl || '/dashboard';
  this._currentUrl = e.urlAfterRedirects;
});
back(fallback: string = '/dashboard'): void {
  this.router.navigate([this._previousUrl || fallback]);
}
```

Without it, every page's back button would have to hardcode the parent URL: `this.router.navigate(['/dashboard/admin'])`. With it, `this.nav.back('/dashboard/admin')` — navigates to the actual previous page (could be admin or investigator dashboard). Especially useful on shared pages like `/dashboard/audit-logs` which is used by both Admin and RegulatoryOfficer roles.

Q47.

What is `pathMatch: 'full'`? When is it needed? **A:** Controls how Angular matches the URL to the route. `pathMatch: 'full'` — the entire URL must match the path (not just the beginning). `pathMatch: 'prefix'` (default) — matches if the URL starts with the path. For `{ path: '', redirectTo: 'dashboard' }` without `pathMatch: 'full'`, Angular would redirect every URL (since every URL starts with ''). With `'full'`, only the exact empty URL redirects.

Q48.

How does Angular pass data between routes? What methods are available? **A:** (1) **Route params** — `/patients/:id` → `route.params['id']`. (2) **Query params** — `/reports?section=kpi` → `route.queryParams['section']`. Your `viewKpiReports()` uses: `this.router.navigate(['/dashboard/reports'], { queryParams: { section: 'reports' } })`. (3) **Route data** — static: `data: { title: 'Documents', roles: ['Admin'] }`. (4) **State** — `navigate(..., { state: { userId: 42 } })` then `this.router.getCurrentNavigation()?.extras.state`. (5) **Service** — store data in a service before navigating, read it after. Your project primarily uses route `data` and services.

Q49.

What is a route resolver? Does your project use one? **A:** A route resolver pre-fetches data before the route activates — the component is rendered only after data is loaded. Implement `Resolve<T>` interface. Useful to avoid "loading" flicker. Your project does NOT use resolvers — all components fetch data in `ngOnInit()` with `loading = true` until complete. Adding resolvers to `AdminProtocolsPageComponent` would pre-fetch protocols before the component is shown.

Q50.

What is `router.events`? How did `AppComponent` use it for page titles? **A:** `router.events` is an Observable that emits all Router events: `NavigationStart`, `NavigationEnd`, `NavigationError`, `GuardsCheckStart`, `ActivationStart`, etc. `AppComponent` subscribes with `.pipe(filter(e => e instanceof NavigationEnd))` — only processes after navigation completes (not during). On each `NavigationEnd`, it reads the deepest route's `data.title` and role to set the browser tab title. Also used by `NavigationService` to track previous URLs.

Topic 5: Template-Driven vs Reactive Forms

Q51.

What are the two types of Angular forms? When do you use each? **A: Template-driven** — form logic is defined in the HTML template using directives (`ngModel`, `ngForm`, `required`). Simple, less code, harder to test. **Reactive** — form logic is defined in the TypeScript class using `FormGroup`, `FormControl`, `Validators`. More code, fully testable, better for complex forms with dynamic fields and custom validators. Your project uses **reactive forms predominantly** — login, register, change password, create/edit modals all use `FormBuilder`. Some inline filters use `[(ngModel)]` (template-driven) for simple bindings.

Q52.

What is `FormBuilder`? How do you use it in your project? **A:** `FormBuilder` is a service that creates `FormGroup` and `FormControl` instances with a concise API. In `LoginComponent`:

```
this.form = this.fb.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', [Validators.required, Validators.minLength(6)]]
});
```

`fb.group({key: [initialValue, validators]})` — first element is the default value, second is a single validator or array. Returns a `FormGroup`. Each key becomes a `FormControl` accessible via `this.form.get('email')` or `this.form.controls['email']`.

Q53.

What are built-in Angular validators? Which ones did your project use? **A:** `Validators.required` — field must not be null/empty. `Validators.email` — must be valid email format. `Validators.minLength(n)` — minimum character count. `Validators.maxLength(n)` — maximum character count. `Validators.min(n)` — numeric minimum. `Validators.max(n)` — numeric maximum. `Validators.pattern(regex)` — regex pattern match. Your project uses all except `pattern` (you wrote custom validators instead). In `RegisterComponent`: `Validators.required`, `Validators.email`, `Validators.minLength(8)`, `Validators.maxLength(256)`.

Q54.

What is a cross-field validator? How did you implement password match validation? **A:** A cross-field validator validates relationships between multiple form controls — not just a single field. In `RegisterComponent`:

```
this.form = this.fb.group(
  { password: [...], confirm: [...] },
  { validators: this.passwordsMatch } // ← applied to the FormGroup, not a control
);

private passwordsMatch(g: FormGroup) {
  const pw = g.get('password')?.value;
  const c = g.get('confirm')?.value;
  return pw && c && pw !== c ? { mismatch: true } : null;
}
```

The validator is on the `FormGroup` level, not the individual controls. Access the cross-field error via `this.form.hasError('mismatch')`. The error shows only when both fields are touched and don't match.

Q55.

What is a custom validator? How did you implement `dobValidator` and `phoneValidator`? **A:** A custom validator is a function that takes `AbstractControl` and returns `ValidationErrors | null`. `null` means valid. `{ errorKey: true }` means invalid.

```
private dobValidator(control: AbstractControl): ValidationErrors | null {
  const dob = new Date(control.value);
  const today = new Date(); today.setHours(0, 0, 0, 0);
  if (dob >= today) return { dobFuture: true };
  const age18Cutoff = new Date(); age18Cutoff.setFullYear(today.getFullYear() - 18);
  if (dob > age18Cutoff) return { dobMinAge: true }; // under 18
  const age120Cutoff = new Date(); age120Cutoff.setFullYear(today.getFullYear() - 120);
  if (dob < age120Cutoff) return { dobMaxAge: true }; // over 120
  return null;
}
```

`phoneValidator` — returns `{ phonePattern: true }` if the value doesn't match `/^[6-9]\d{9}$/`. Clinical context: patients must be 18+ (adult consent for trials), DOB cannot be today or future. India-specific mobile validation.

Q56.

What is `this.form.markAllAsTouched()`? Why do you call it on submit? **A:** Marks all controls (and child controls) as "touched".

Angular only shows validation errors on touched controls — this prevents showing errors before the user has interacted. When a user clicks Submit without filling anything, calling `markAllAsTouched()` immediately marks all fields as touched, causing all validation error messages to appear. Your `LoginComponent.submit()`:

```
if (this.form.invalid) { this.form.markAllAsTouched(); return; }
```

This replaces having to call `control.markAsTouched()` on every individual control.

Q57.

What is `form.patchValue()` vs `form.setValue()`? Where does your project use `patchValue`? **A:** `setValue({...})` — sets ALL form controls. Throws an error if a key is missing. `patchValue({...})` — sets ONLY the specified controls, ignores others — safer for partial updates. In `openEditModal(p)` in `AdminProtocolsPageComponent`:

```
this.editForm.patchValue({
  title: p.title,
  startDate: this.toDateInput(p.startDate),
  endDate: p.endDate ? this.toDateInput(p.endDate) : '',
  status: p.status
});
```

Used to pre-populate the edit form with the current item's values before showing the modal. `patchValue` is used because the form might have extra controls not in the item.

Q58.

What is `FormGroup.reset()`? What does `reset({ status: 'Pending' })` do? **A:** `reset()` — resets all controls to their initial values and marks them as untouched/pristine/unused. `reset({ status: 'Pending' })` — resets all controls AND sets specific controls to given values. In `openCreateModal()`:

```
this.createForm.reset({ status: 'Pending' });
```

Clears all fields and sets `status` to `'Pending'` as the default. Without the argument, `status` would be `null` after reset. This ensures the modal always opens in a clean state with sensible defaults.

Q59.

What is the difference between `form.value` and `form.getRawValue()`? **A:** `form.value` — returns values of enabled controls only. Disabled controls are excluded. `form.getRawValue()` — returns ALL controls including disabled ones. If you have a `status` field disabled in the edit form (to show the current status but not allow changing it), `form.value` would exclude it but you'd still need it for the PUT request. Use `getRawValue()` for that case. Your project uses `this.form.value` — no disabled controls.

Q60.

What does `Validators.required` check exactly? Does it validate whitespace? **A:** `Validators.required` returns an error if the value is `null`, `undefined`, empty string `''`, or empty array `[]`. It does NOT validate whitespace — `' '` passes `required`. That's why your submit methods do `v.name.trim()` before sending to the backend. For whitespace validation you'd use a custom validator checking `value?.trim().length > 0`.

Q61.

What is form state? What are `valid`, `invalid`, `touched`, `untouched`, `dirty`, `pristine`? **A:** `valid` — all validators pass. `invalid` — at least one validator fails. `touched` — user has focused and then left the control (`blur` event). `untouched` — user hasn't interacted. `dirty` — value has been changed by user. `pristine` — value hasn't been changed since initialization. Your templates show error messages with patterns like `if (email.touched && email.errors?.['required'])`. You use `if (this.form.invalid)` before submit.

Q62.

What is `dateRangeValidator` in `AdminProtocolsPageComponent`? Why is it a standalone function? **A:**

```
function dateRangeValidator(group: AbstractControl): ValidationErrors | null {
  const start = group.get('startDate')?.value;
  const end = group.get('endDate')?.value;
  if (start && end && end < start) return { dateRange: 'End date must be on or after start date.' };
  return null;
}
```

Applied to the entire `FormGroup`: `this.fb.group({ ... }, { validators: dateRangeValidator })`. It's a standalone function (not a class method) because it doesn't need `this` context and is reusable without binding. It's a cross-field validator — checks that `endDate >= startDate`. The error message is the string value, not just `true`.

Q63.

How does `valueChanges` Observable work? Where is it used in `AdminProtocolsPageComponent`? **A:** `formControl.valueChanges` is an

Observable that emits every time the control's value changes. Used to sync dependent form controls:

```
this.editForm.get('startDate')!.valueChanges.subscribe(syncStatus);
this.editForm.get('endDate')!.valueChanges.subscribe(syncStatus);
```

`syncStatus()` recalculates which status options are valid based on the dates and auto-updates the status dropdown. When the admin changes the start date, the allowed status list updates instantly. This is reactive programming applied to forms.

Q64.

What is template-driven form validation vs reactive? Which approach did the `InvestigatorAdverseEventsPageComponent` use? **A:**

`InvestigatorAdverseEventsPageComponent` uses a **plain object** as the form model (not `FormGroup`):

```
form = { patientID: '', protocolID: '', description: '', severity: '', reportedDate: '' };
```

And a manual `validateForm()` method:

```
validateForm(): boolean {
  this.formErrors = {};
  if (!this.form.patientID) this.formErrors['patientID'] = 'Patient is required.';
  if (this.form.description.trim().length < 10) this.formErrors['description'] = '...';
  return Object.keys(this.formErrors).length === 0;
}
```

This is neither template-driven nor reactive — it's manual JavaScript validation. Works but is harder to scale. The `AdminProtocolsPageComponent` uses full `ReactiveFormsModule` — cleaner and more maintainable.

Q65.

What is `finalize()` RxJS operator? How is it used in the login form? **A:** `finalize(callback)` runs the callback when the Observable completes OR errors — equivalent to a `finally` block. In `LoginComponent.submit()`:

```
this.auth.login(this.form.value).pipe(
  finalize(() => this.loading = false) // ← always runs, success or error
).subscribe({
  next: () => this.router.navigate(['/dashboard']),
  error: err => { this.error = '...' }
});
```

`this.loading = false` runs regardless of success or failure. Without `finalize`, you'd need to set `loading = false` in both `next` and `error` callbacks, risking a missed `error` path leaving the button stuck in loading state.

Topic 6: HTTP Client & Interceptors

Q66.

What is `HttpClient` in Angular? How is it injected and used? **A:** `HttpClient` is Angular's HTTP service for making REST API calls. Returns Observables (not Promises). Must import `HttpClientModule` in `AppModule` (done). Injected in the constructor: `constructor(private http: HttpClient) {}`. Usage in your project:

```
this.http.get<any>(`${environment.apiUrl}/protocols?pageSize=200`).subscribe(res => { ... });
this.http.post(`${environment.apiUrl}/sites`, body).subscribe({ next: ..., error: ... });
this.http.put(`${environment.apiUrl}/protocols/${id}`, body).subscribe(...);
this.http.delete(`${environment.apiUrl}/sites/${id}`).subscribe(...);
```

Q67.

What is an HTTP interceptor? Explain your `AuthInterceptor` in detail. **A:** An interceptor is middleware that intercepts every HTTP request/response. Your `AuthInterceptor`:

```
intercept(req: HttpRequest<unknown>, next: HttpHandler): Observable<HttpEvent<unknown>> {
  const token = this.auth.getToken();
  const cloned = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req; // no token for public endpoints

  return next.handle(cloned).pipe(
    catchError((err: HttpResponse) => {
      if (err.status === 401) {
        if (req.url.includes('/auth/login')) return throwError(() => err); // let login handle its own 401
        if (!AuthInterceptor.logoutInFlight) {
          AuthInterceptor.logoutInFlight = true;
          this.auth.logout();
          queueMicrotask(() => { AuthInterceptor.logoutInFlight = false; });
        }
        return EMPTY; // swallow the 401 - component doesn't need to handle it
      }
      return throwError(() => err); // re-throw all other errors
    })
  );
}
```

Every outgoing request automatically gets `Authorization: Bearer <token>`. 401 responses (expired token) automatically log out and redirect to login.

Q68.

What is `req.clone()`? Why can't you modify `req` directly? **A:** `HttpRequest` is immutable — you cannot mutate it. `req.clone({ ... })` creates a new request with specified overrides. This is by design: interceptors don't have side effects on the original request. `req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })` creates a copy with the auth header added. The original `req` is unchanged. This is the interceptor pattern — each interceptor gets the original and passes a modified clone forward.

Q69.

What is the static `logoutInFlight` flag in `AuthInterceptor`? Why is it static? **A:** When a dashboard page with 8 parallel `forkJoin` requests all get 401 (token expired simultaneously), all 8 would call `this.auth.logout()` — causing 8 redirects and race conditions. `private static logoutInFlight = false` — static means it's shared across ALL `AuthInterceptor` instances. The first 401 sets it to `true` and calls `logout()`. Subsequent 401s see `logoutInFlight === true` and skip. `queueMicrotask(() => logoutInFlight = false)` resets it on the next microtask (after the redirect has dispatched).

Q70.

What is `EMPTY` observable? Why return it for 401 responses? **A:** `EMPTY` is an Observable that completes immediately without emitting any values. Returning `EMPTY` from `catchError` means: the error is swallowed, the subscriber's `error` callback is NOT called, the subscription completes silently. Why: when a 401 triggers logout, the component that made the request is about to be destroyed (navigating away). If the component's `error` callback ran, it might try to set `this.error = 'Unauthorized'` on a component being destroyed — causing errors or confusing UI flicker. `EMPTY` silently absorbs the 401 since the session is ending anyway.

Q71.

What is `throwError(() => err)`? Why the function wrapper? **A:** `throwError(errorFactory)` creates an Observable that immediately errors. The function wrapper `() => err` is required in RxJS 7+ (your project uses recent RxJS). In older RxJS: `throwError(err)`. The function is called lazily when subscribed — preventing eager error creation. When your interceptor re-throws non-401 errors: `return throwError(() => err)`, the component's `error` callback receives the `HttpResponse` and

can show the error message.

Q72.

What is `forkJoin`? How does `AdminDashboardComponent` use it? **A:** `forkJoin([obs1, obs2, ...])` or `forkJoin({key1: obs1, key2: obs2})` runs all Observables **in parallel** and emits a single value when ALL complete — combining their results. In `AdminDashboardComponent.ngOnInit()`:

```
forkJoin({
  users:      this.ds.count('users'),
  activeUsers: this.ds.count('users', { isActive: 'true' }),
  protocols:   this.ds.count('protocols'),
  ...10 total
}).pipe(takeUntil(this.destroy$)).subscribe(d => {
  this.users      = d.users;
  this.protocols  = d.protocols;
  this.loading    = false;
});
```

All 10 API calls run in parallel (not sequentially). When all 10 complete, the object `d` contains all results. Sequential would take 10× longer. If any Observable errors, `forkJoin` errors immediately.

Q73.

What is the `DashboardService.count()` method? Why `page=1&pageSize=1`? **A:**

```
count(resource: string, params: Record<string, string> = {}): Observable<number> {
  const qs = new URLSearchParams({ page: '1', pageSize: '1', ...params });
  return this.http.get<PagedResult<unknown>>(`${this.api}/${resource}?${qs}`)
    .pipe(
      timeout(REQUEST_TIMEOUT_MS),
      map(r => r?.totalCount ?? 0),
      catchError(err => { console.error(...); return of(0); })
    );
}
```

`pageSize: '1'` — fetches only 1 item but the API response includes `totalCount` (the total count across all pages). So you get the count without fetching all records. `timeout(8000)` — if the downstream service is slow/down, fails after 8 seconds. `catchError(() => of(0))` — if any count fails, shows 0 rather than crashing the entire dashboard.

Q74.

What is `HttpErrorResponse`? How does your project handle API errors? **A:** `HttpErrorResponse` is the error type for failed HTTP requests. It has: `status` (HTTP status code like 400/401/404/500), `error` (the response body — your `ErrorDetail` JSON), `message` (Angular's error description). Your components extract the user-friendly message:

```
error: err => {
  this.error =
    err.error?.errors?.[0] ?? // array of errors
    err.error?.message ?? // message field
    err.error?.error ?? // your ErrorDetail.Error field
    'Invalid email or password. Please try again.';
}
```

The chain `??` tries each field and falls back to a default if all are null. This handles different error response shapes from different endpoints.

Q75.

What is `URLSearchParams`? How is it used to build query strings? **A:** `URLSearchParams` is a browser-native API for building URL query strings. In `AdminProtocolsPageComponent.loadProtocols()`:

```
const qs = new URLSearchParams({ page: String(page), pageSize: String(this.listPageSize) });
if (this.searchTerm.trim()) qs.set('search', this.searchTerm.trim());
if (this.filterStatus) qs.set('status', this.filterStatus);
this.http.get<any>(`${environment.apiUrl}/protocols?${qs}`)
```

`qs.toString()` produces `page=1&pageSize=10&search=covid&status=Active`. Used instead of manual string concatenation for proper URL encoding.

Topic 7: RxJS & Observables

Q76.

What is an Observable? How is it different from a Promise? **A:** An Observable is a lazy data stream that can emit 0 or more values over time. A Promise resolves once and is eager (starts immediately). Key differences: **Observable** — lazy (starts only when subscribed), cancellable (`unsubscribe()`), can emit multiple values, supports operators (`map`, `filter`, `switchMap`). **Promise** — eager (starts immediately), not cancellable, resolves once. `HttpClient` returns Observables. Your `AuthService.login()` returns `Observable<AuthResponse>`. Subscribed in `LoginComponent.submit()`.

Q77.

What is the `subscribe()` method? What are the three callbacks? **A:** `subscribe({ next, error, complete })` — triggers the Observable to start. `next(value)` — called for each emitted value. `error(err)` — called when the Observable errors (stops after error). `complete()` — called when the Observable completes normally (no more values). Your `forkJoin({...}).subscribe({ next: d => { this.users = d.users; this.loading = false; }, error: () => { this.loading = false; } })`. HTTP requests complete automatically after emitting one response.

Q78.

What is `pipe()` in RxJS? What operators did your project use? **A:** `pipe()` chains operators that transform the Observable stream without modifying the source. Operators used in your project: `map()` — transforms emitted values. `catchError()` — handles errors. `filter()` — filters events (e.g., only `NavigationEnd`). `takeUntil()` — completes the Observable when another emits. `timeout()` — errors if no emission within time. `tap()` — side effects without transforming (stores token in `AuthService.login()`). `finalize()` — runs callback on completion or error.

Q79.

What is `takeUntil` and the `destroy$` pattern? Why is it better than `unsubscribe()`? **A:** `takeUntil(notifier$)` completes the Observable when `notifier$` emits. The `destroy$` pattern:

```
private destroy$ = new Subject<void>();

ngOnInit() {
  this.http.get<any>(...).pipe(takeUntil(this.destroy$)).subscribe(...);
  forkJoin({...}).pipe(takeUntil(this.destroy$)).subscribe(...);
}

ngOnDestroy(): void {
  this.destroy$.next(); // emit to complete all subscriptions
  this.destroy$.complete(); // clean up the Subject itself
}
```

Better than `unsubscribe()` because: one `destroy$.next()` cancels ALL subscriptions in the component, not one by one. You can't forget to cancel a subscription. Clean and scalable.

Q80.

What is `BehaviorSubject`? How does `AuthService` use it for user state? **A:** `BehaviorSubject<T>` is a Subject that: (1) holds the current value, (2) immediately emits the current value to new subscribers, (3) emits future values to all subscribers. In `AuthService`:

```
private currentUserSubject = new BehaviorSubject<UserInfo | null>(this.getStoredUser());
currentUser$ = this.currentUserSubject.asObservable();
```

Initialized with the stored user from `localStorage`. When login succeeds: `this.currentUserSubject.next(res.user)`. When logout: `this.currentUserSubject.next(null)`. Any component that subscribes to `currentUser$` immediately gets the current user AND is notified of future changes. `asObservable()` exposes it as a read-only Observable — external code cannot call `.next()` on it.

Q81.

What is `tap()` operator? How did `AuthService.login()` use it? **A:** `tap(sideEffect)` performs a side effect for each emission without modifying the value — like a transparent wire tap. In `AuthService.login()`:

```
return this.http.post<AuthResponse>(url, req).pipe(
  tap(res => {
    localStorage.setItem(this.TOKEN_KEY, res.token);
    localStorage.setItem(this.USER_KEY, JSON.stringify(res.user));
    this.currentUserSubject.next(res.user);
  })
);
```

The HTTP call returns `Observable<AuthResponse>`. `tap` stores the token and user without changing the emitted `AuthResponse`. The caller (`LoginComponent.submit()`) subscribes and gets the full `AuthResponse` — unmodified.

Q82.

What is `of()` operator? Where did your project use it as a fallback? **A:** `of(value)` creates an Observable that immediately emits the given value and completes. Used as a fallback in `catchError`:

```
catchError(err => { console.error(...); return of(0); }) // in DashboardService.count()
catchError(() => of({ items: [] })) // in NotificationsPageComponent
```

When an HTTP call fails, instead of propagating the error, return a safe default value. The component still gets a usable value (0 for counts, empty array for lists) and can display gracefully.

Q83.

What is the difference between `Subject`, `BehaviorSubject`, `ReplaySubject`, and `AsyncSubject`? **A:** `Subject` — multicasts, no initial value, late subscribers miss past emissions. `BehaviorSubject(initialValue)` — holds current value, emits it immediately to new subscribers. `ReplaySubject(n)` — replays the last `n` emissions to new subscribers. `AsyncSubject` — only emits the last value when the Observable completes. Your project uses `BehaviorSubject` for `currentUserSubject` (needs current value on subscription) and `Subject<void>` for `destroy$` (only needs to emit once as a signal).

Q84.

What is `catchError` operator? What happens after it returns an Observable? **A:** `catchError(handler)` intercepts an error, runs `handler(error)`, and the returned Observable becomes the new stream. The original error is swallowed. In `DashboardService`:

```
catchError(err => { console.error(...); return of(0); })
```

The stream that errored is replaced by `of(0)`, which emits `0` and completes normally. The subscriber's `error` callback is never called — it receives `0` via `next`. In `AuthInterceptor`, `catchError` returns `EMPTY` for 401s — the stream completes without emitting.

Q85.

What is `timeout()` operator? Why did `DashboardService` use 8 seconds? **A:** `timeout(milliseconds)` errors the Observable if no emission arrives within the specified time. After 8000ms: `TimeoutError` is thrown, caught by `catchError`, returns `of(0)` or `of([])`. Why 8 seconds: if a downstream microservice is down or slow, the dashboard shouldn't wait forever showing a spinner. 8 seconds is enough for a healthy service but short enough to give a timely fallback.

Q86.

What is `map()` operator? How is it used in `DashboardService.count()`? **A:** `map(transform)` transforms each emitted value. In `DashboardService.count()`:

```
map(r => r?.totalCount ?? 0)
```

The HTTP call emits `PagedResult<unknown>`. `map` extracts just `totalCount` from it. The subscriber receives a `number` directly, not the full paged result object. This abstracts the response shape — callers don't need to know about the `PagedResult` wrapper.

Q87.

What is the `filter()` operator? Where is it used in your project? **A:** `filter(predicate)` only allows values through when `predicate` returns `true`. In `AppComponent.ngOnInit()`:

```
this.router.events.pipe(filter(e => e instanceof NavigationEnd)).subscribe(...)
```

`router.events` emits many types: `NavigationStart`, `GuardsCheckStart`, `ActivationStart`, `NavigationEnd`, etc. `filter` passes only `NavigationEnd` events — fires only when navigation is complete and the component is rendered. Also in `NavigationService`: same pattern.

Q88.

What does `takeUntil(this.destroy$)` do if the component completes before `destroy$` emits? **A:** If the HTTP call completes (emits one response) before `destroy$` emits, `takeUntil` has no effect — the Observable has already completed naturally. `takeUntil` only cancels ONGOING subscriptions. For short-lived single HTTP calls, it makes no practical difference but is still good practice — if the user navigates away while the call is in-flight, `takeUntil` cancels it.

Q89.

What is the difference between cold and hot Observables? **A:** **Cold** — the data producer starts inside the Observable, separately per subscriber. Each subscriber gets its own execution. `HttpClient.get()` is cold — each `.subscribe()` makes a

new HTTP request. `of()`, `from()` are cold. **Hot** — the data producer exists outside; all subscribers share the same stream. `fromEvent(button, 'click')` is hot. `BehaviorSubject.asObservable()` is hot. `router.events` is hot. Your `currentUser$` is hot — multiple components can subscribe and all see the same emissions.

Q90.

What is the async pipe? Why is it better than manual subscribe/unsubscribe? **A:** `{{ observable$ | async }}` in templates: subscribes to the Observable when the component is created, updates the view when a new value arrives, and automatically unsubscribes when the component is destroyed. Your project doesn't use `async pipe` — all services return data via `subscribe()` and store results in component properties (`this.protocols = data`). Using `async pipe` would eliminate `destroy$` boilerplate for Observable bindings.

Topic 8: Services, Dependency Injection & State

Q91.

What is a service in Angular? What is `providedIn: 'root'`? **A:** A service is a TypeScript class decorated with `@Injectable()` that provides business logic, data fetching, or shared state — not tied to a specific component. `providedIn: 'root'` registers the service in the root injector — it becomes a singleton available application-wide without needing to add it to any module's `providers`. Your `AuthService`, `DashboardService`, `NavigationService`, `AuthGuard`, `RoleGuard` all use `providedIn: 'root'`.

Q92.

How is `AuthService` used as a state management solution? **A:** `AuthService` holds the authenticated user state using `BehaviorSubject<UserInfo | null>`. When the user logs in → `currentUserSubject.next(user)`. Components access `auth.currentUser` (synchronous) or subscribe to `auth.currentUser$` (reactive). The token is persisted in `localStorage`. On app start, `getStoredUser()` restores the state from `localStorage`. This is a lightweight in-app state management solution without NgRx.

Q93.

What is the difference between `providedIn: 'root'` and module-level providers? **A:** `providedIn: 'root'` — singleton, tree-shakeable (removed if not used), application-wide. Module-level (`providers: [MyService]` in `NgModule`) — a new instance per module injector; if a lazy-loaded module provides a service, it gets its own instance separate from root. Module-level providers are NOT tree-shakeable. For your use case — `AuthService` needed by all components across all modules — `providedIn: 'root'` is correct.

Q94.

How does DI injection work in Angular services vs components? **A:** Both use constructor injection. Services: `@Injectable({ providedIn: 'root' }) export class DashboardService { constructor(private http: HttpClient) {} }`. Components: `export class LoginComponent { constructor(private fb: FormBuilder, private auth: AuthService, private router: Router) {} }`. The Angular DI framework reads constructor parameter types (TypeScript metadata emitted with `emitDecoratorMetadata`), looks up registered providers by type, and injects instances. You never call `new LoginComponent(...)` — Angular does.

Q95.

What is `localStorage` in the context of Angular? How does `AuthService` use it? **A:** `localStorage` is a browser-native key-value store that persists across page refreshes (unlike `sessionStorage` which clears on tab close). `AuthService` stores: `lt_token` (JWT string) and `lt_user` (JSON-serialized `UserInfo`). On app start, `getStoredUser()` reads from `localStorage` to restore the user session. On logout, both keys are removed. **Security note:** `localStorage` is accessible by JavaScript — XSS attacks can read it. The alternative (HttpOnly cookies) is more secure but requires server-side cookie handling. Your project's choice (`localStorage`) is common but noted as a trade-off.

Topic 9: Pipes, Directives & Templates

Q96.

What are Angular pipes? What built-in pipes did your project use? **A:** Pipes transform values in templates: `{{ value | pipeName }}`. Built-in pipes in your project: `DatePipe` — `{{ date | date:'mediumDate' }}` formats dates. `AsyncPipe` — (not used but important). `UpperCasePipe` / `LowerCasePipe` — (not used explicitly). `CurrencyPipe`, `DecimalPipe`, `PercentPipe` — useful for KPI reports. `JsonPipe` — debugging. Your `AppModule` configures `DATE_PIPE_DEFAULT_OPTIONS` to use IST timezone globally — every `DatePipe` usage formats in India Standard Time.

Q97.

What is `*ngIf`? How do you use it with an else template? **A:** `*ngIf="condition"` renders the element only when condition is truthy. `*ngIf="loading; else contentBlock"` with `<ng-template #contentBlock>` renders the else block when loading is false. Your components use: `*ngIf="loading"` to show a spinner, `*ngIf="errorMsg"` to show an error panel, `*ngIf="documents.length === 0"` to show the empty state component. The `*` is syntactic sugar — `*ngIf="x"` expands to `[ngIf]="x"` on an `<ng-template>`.

Q98.

What is `*ngFor`? What are the exported variables? **A:** `*ngFor="let item of items; trackBy: trackFn; let i = index; let last = last"`. Exported variables: `index` (0-based position), `first`, `last`, `even`, `odd` (booleans). Your pages use: `*ngFor="let p of protocolList"` to render protocol rows, `*ngFor="let page of pageNumbers"` to render pagination buttons. `trackBy` function improves performance by telling Angular which items changed (by key) instead of re-rendering all DOM nodes.

Q99.

What is `NgClass` and `NgStyle`? How are they used for status badges? **A:** `[ngClass]="expression"` — dynamically applies CSS classes. `[ngStyle]="expression"` — dynamically applies inline styles. In your components, badge classes are applied via `ngClass`:

```
<span [ngClass]="statusClass(p.status)">{{ p.status }}</span>
```

Where `statusClass(s)` returns `'badge-green'`, `'badge-amber'`, `'badge-red'` etc. based on the status string. This keeps styling logic in the component (testable) rather than in the template.

Q100.

What is `[(ngModel)]` two-way data binding? Where did your project use it? **A:** `[(ngModel)]` binds a form input to a component property bidirectionally — `[ngModel]` (property binding) + `(ngModelChange)` (event binding) combined. `[(x)]` is the "banana in a box" syntax. Your notification page, filter dropdowns, and search inputs use: `[(ngModel)]="filterStatus"`, `[(ngModel)]="searchTerm"`. When the user types, `searchTerm` is updated. When the component sets `searchTerm = ''`, the input clears. Requires `FormsModule` to be imported.

Q101.

What is the `async` pipe and why is it powerful for avoiding memory leaks? **A:** `{{ obs$ | async }}` subscribes in the template. Key advantage: automatically unsubscribes when the component is destroyed — no `destroy$` Subject needed. Can be used with `*ngIf` for null-guarding: `*ngIf="user$ | async as user"`. Your project uses manual subscriptions, but converting `currentUser$` to `async` pipe usage would eliminate manual unsubscription boilerplate.

Q102.

What is `*ngSwitch`? Where would you use it in LifeTrack? **A:** `*ngSwitch="expression"` with `*ngSwitchCase="value"` and `*ngSwitchDefault` renders the matching case. Useful for rendering different components per role:

```
<ng-container *ngSwitch="user?.role">
  <app-admin-dashboard *ngSwitchCase="'Admin'"></app-admin-dashboard>
  <app-ctm-dashboard *ngSwitchCase="'ClinicalTrialManager'"></app-ctm-dashboard>
  <app-patient-dashboard *ngSwitchDefault></app-patient-dashboard>
</ng-container>
```

Your project uses separate routes per role (preferred) rather than `*ngSwitch`.

Q103.

What is the `title` service used in `AppComponent`? **A:** `Title` from `@angular/platform-browser` allows setting the browser tab's `<title>` tag programmatically. `this.titleService.setTitle('Documents - Regulatory Officer | LifeTrack')`. Your `AppComponent.ngOnInit()` sets the title on every `NavigationEnd` event — combining the route's `data.title` (e.g., "Documents") with the user's role label (e.g., "Regulatory Officer"). Improves UX (users see meaningful tab titles) and SEO.

Q104.

What is property binding `[property]` vs event binding `(event)` vs two-way `[(ngModel)]`? **A:** `[href]="url"` — one-way from

component to DOM. `(click)="handler()"` — DOM event to component method. `[(ngModel)]="searchTerm"` — two-way binding. `{{ expression }}` — interpolation (also one-way component to DOM). In your templates: `[disabled]="form.invalid"` disables submit button when form invalid (property binding). `(click)="openCreateModal()"` opens modal on click (event binding). `[(ngModel)]="filterStatus"` keeps filter dropdown in sync with component (two-way).

Topic 10: TypeScript in Angular

Q105.

What are TypeScript interfaces? How are they used in your project? **A:** Interfaces define the shape of objects without creating runtime code (erased at compile). Your `auth.models.ts`:

```
export interface UserInfo {
  userID: number;
  name: string;
  email: string;
  role: string;
}
export interface AuthResponse {
  token: string;
  expiresAtUtc: string;
  user: UserInfo;
}
```

Type-safe HTTP responses: `this.http.post<AuthResponse>(url, req)`. TypeScript checks that `res.user.role` exists at compile time. `PagedResult<T>` is a generic interface used across all list responses.

Q106.

What are TypeScript generics? How is `PagedResult<T>` used? **A:** Generics allow writing type-safe code that works with multiple types. `PagedResult<T>` — a generic interface where `T` is the item type:

```
interface PagedResult<T> {
  page: number; pageSize: number; totalCount: number; totalPages: number;
  items: T[];
}
```

Usage: `this.http.get<PagedResult<Protocol>>('url')` — TypeScript knows `items` is `Protocol[]`. `this.http.get<PagedResult<unknown>>('url').pipe(map(r => r.totalCount))` — used in `DashboardService.count()` when you only need the count, not typed items.

Q107.

What is `Record<K, V>` in TypeScript? Where does your project use it? **A:** `Record<K, V>` is a utility type for an object with keys of type `K` and values of type `V`. In your project:

```
protocolMap: Record<number, string> = {}; // AdminAssignmentsPageComponent
siteMap: Record<number, string> = {};
investigatorMap: Record<number, string> = {};
```

These lookup maps translate IDs to display names (e.g., `protocolMap[5] = "COVID-19 Trial"`). Also: `const ROLE_LABELS: Record<string, string> = { Admin: 'Admin', ClinicalTrialManager: 'Clinical Trial Manager', ... }.`

Q108.

What is optional chaining `?.` in TypeScript? What is `??` (nullish coalescing)? **A:** `?.` — short-circuits to `undefined` if the left side is `null` / `undefined`. `user?.name?.split('')[0]` — won't throw if `user` is null or `name` is undefined. `??` — returns the right side only if the left is `null` or `undefined` (NOT for falsy values like `0` or `''`). `r?.totalCount ?? 0` — if `r` is null, or `totalCount` is undefined/null, returns `0`. `err.error?.error ?? err.error?.message ?? 'Default'` — the error extraction chain in all your components.

Q109.

What are type assertions in TypeScript (`as` keyword)? Where are they used? **A:** `value as Type` — tells TypeScript "trust me, this is of this type." Used when TypeScript can't infer the type. In `AuthInterceptor`: `(e as NavigationEnd).urlAfterRedirects` — `router.events` emits `Event` type, but we've filtered to only `NavigationEnd`, so the assertion is safe. `(e.target as HTMLElement).classList.contains('sub-backdrop')` — `event.target` is `EventTarget`, casting to `HTMLElement` for DOM access.

Q110.

What is a union type? How are they used in your project? **A:** A union type allows a value to be one of several types: `string | null | undefined`. In `AuthService`: `BehaviorSubject<UserInfo | null>` — can hold either a `UserInfo` or `null`. In `AuditEntry` interface: `changedByUserID: number | null`. In `AdminAuditLogsPageComponent`: `activeTab: 'entries' | 'logins'` — constrained string union, TypeScript would error if you assigned `'other'`. This prevents typos and enables exhaustive type checking.

Topic 11: Security Patterns in the Frontend

Q111.

Why does `RoleGuard` read the role from the JWT token instead of `localStorage`? **A:** The comment in `role.guard.ts` explains directly: "The user object in `localStorage` is tamperable — a malicious user could edit `role: 'Patient'` to `role: 'Admin'` in DevTools to bypass the client-side `RoleGuard`." Reading from the JWT: `this.auth.getRoleFromToken()` — the token's signature is checked by the backend on every request. Even if someone tampered with `localStorage`, the JWT claims cannot be modified without invalidating the signature. The JWT is the authoritative source for roles in the frontend too.

Q112.

How does `AuthService.decodeToken()` work? Why not use a JWT library? **A:**

```
private decodeToken(): Record<string, any> | null {
  const parts = token.split('.');
  if (parts.length !== 3) return null;
  const payload = parts[1].replace(/-/g, '+').replace(/_/g, '/');
  const padded = payload + '='.repeat((4 - payload.length % 4) % 4);
  const json = atob(padded);
  return JSON.parse(json);
}
```

A JWT has three Base64url-encoded parts. Base64url differs from Base64: `-` replaces `+`, `_` replaces `/`, no padding `=`. `atob()` is the browser's Base64 decoder. Padding is added to make the length a multiple of 4. No library needed — native browser APIs decode the payload. Note: this only reads the payload — it does NOT verify the signature (that's the backend's job).

Q113.

What is `isLoggedIn()` and why does it check token expiry client-side? **A:**

```
isLoggedIn(): boolean {
  const token = this.getToken();
  if (!token) return false;
  const exp = this.getTokenExpiry();
  return exp ? exp > Date.now() : true;
}
```

Checks both token existence AND expiry. `getTokenExpiry()` reads the `exp` claim from the JWT (seconds since epoch) and multiplies by 1000 for milliseconds. `exp > Date.now()` — true if not yet expired. Purpose: prevents the UI from showing as logged-in with an expired token before the first API call returns 401. The `AuthGuard` uses this — if the token expired, route is blocked before any API calls are made.

Q114.

What is XSS? How could it affect your token storage? **A:** Cross-Site Scripting (XSS) — an attacker injects malicious JavaScript into your page (via user input rendered unsafely, or a compromised CDN script). Malicious JS can read `localStorage.getItem('lt_token')` and send the JWT to an attacker's server. Angular has built-in XSS protection — all template interpolation (`{{ }}`) and property bindings (`[innerHTML]`) are sanitized. But third-party scripts could still steal `localStorage` tokens. The more secure alternative is `HttpOnly` cookies (not readable by JavaScript), but requires backend cookie handling.

Q115.

What is CSRF? Is it relevant for your project? **A:** Cross-Site Request Forgery — malicious site tricks the browser into making authenticated requests to your app. Cookies are sent automatically by browsers, making cookie-based auth vulnerable. JWT in `Authorization` header (your approach) is NOT sent automatically by cross-site requests — CSRF is largely irrelevant for `Authorization` header-based JWT auth. This is one advantage of your bearer token approach over cookies.

Q116.

Why is the backend authorization the "ultimate source of truth" even with `RoleGuard`? **A:** The comment in `role.guard.ts` states it explicitly: "Backend authorization is still the ultimate source of truth — this guard just prevents the wrong UI from rendering for the wrong role." A determined user could: disable JavaScript guards, use the API directly with Postman using a Patient token. Backend `RoleAuthorizeFilter` always validates the JWT role before executing any action — the frontend guard is purely for UX (preventing wrong dashboards from loading), not security.

Topic 12: Performance & Build

Q117.

What is debouncing? Where does your project use it for search? **A:** Debouncing delays execution until a pause in user input — prevents an API call on every keystroke. In `AdminProtocolsPageComponent` :

```
private searchTimer: any;
onSearch(value: string) {
  this.searchTerm = value;
  clearTimeout(this.searchTimer);
  this.searchTimer = setTimeout(() => this.loadProtocols(1), 400); // 400ms debounce
}
ngOnDestroy(): void { clearTimeout(this.searchTimer); }
```

Without debouncing, typing "COVID-19" would make 7 API calls. With 400ms debounce: only one call fires after the user stops typing. `clearTimeout` in `ngOnDestroy` prevents the pending timer from firing after the component is destroyed.

Q118.

What is `trackBy` in `*ngFor` and why is it a performance optimization? **A:** Without `trackBy`, when the data array changes, Angular re-renders ALL DOM nodes for `*ngFor`. With `trackBy: trackByFn`, Angular compares items by the return value of `trackByFn`. Only changed items are re-rendered.

```
trackByProtocol(index: number, p: Protocol): number { return p.protocolID; }
```

Template: `*ngFor="let p of protocolList; trackBy: trackByProtocol"`. When one protocol is updated via `this.protocolList[idx] = updated`, only that row's DOM is updated. Your project doesn't implement `trackBy` — adding it to all `*ngFor` lists would be a performance improvement especially for the audit logs page (100+ rows).

Q119.

What is AOT and how does it improve production performance? **A:** AOT (Ahead-of-Time compilation) compiles Angular templates at build time, not in the browser. Benefits: (1) Smaller bundle — Angular compiler not shipped to the browser. (2) Faster rendering — templates already compiled to JavaScript functions. (3) Earlier error detection — template errors caught at build time. (4) Security — expressions evaluated at compile time, not at runtime (reduces injection attacks). Your production build uses AOT automatically (`ng build --configuration=production`).

Q120.

What are Angular build budgets? Where are they in your project? **A:** In `angular.json` under `configurations.production` :

```
"budgets": [
  { "type": "initial", "maximumWarning": "500kB", "maximumError": "1MB" },
  { "type": "anyComponentStyle", "maximumWarning": "8kB", "maximumError": "40kB" }
]
```

If the initial bundle exceeds 500kB — warning. Exceeds 1MB — build error. Component styles exceeding 8kB/40kB — warning/error. Enforces performance discipline. Your app's lazy loading keeps the initial bundle small. Component styles should stay lean.

Topic 13: Project-Specific Deep Dives

Q121.

Explain the complete login flow from button click to dashboard render. **A:** (1) User fills email + password in `LoginComponent`. (2) `form.markAllAsTouched()` triggers any validation errors. (3) `this.auth.login(this.form.value)` called — `AuthService.login()` fires `POST /gateway/auth/login`. (4) `AuthInterceptor.intercept()` runs — no token yet, sends request without header. (5) Backend validates credentials, returns `{ token, expiresAtUtc, user }`. (6) `AuthService.login()` `tap()` stores token in `localStorage`, calls `currentUserSubject.next(user)`. (7) Observable completes, `LoginComponent.next` runs: `this.router.navigate(['/dashboard'])`. (8) `AuthGuard.canActivate()` runs — `isLoggedIn()` returns true. (9) `DashboardModule` lazy-loads. (10) `RoleHomeComponent.ngOnInit()` reads `auth.currentUser.role`, navigates to `/dashboard/admin`. (11) `AdminDashboardComponent` renders, `ngOnInit()` fires `forkJoin({10 API calls})`. (12) All calls go through interceptor with JWT header. (13) Stats populate.

Q122.

What is `RoleHomeComponent` and why does it have an empty template? **A:** `RoleHomeComponent` is the default dashboard child route (path `''`). Its only purpose is to read the user's role and navigate to the role-specific dashboard:

```
ngOnInit() {
  const role = this.auth.currentUser?.role ?? '';
  const target = ROLE_ROUTES[role]; // e.g., '/dashboard/admin'
  if (target) this.router.navigate([target], { replaceUrl: true });
}
```

It has `template: ''` (empty) because it never renders — it immediately redirects. `replaceUrl: true` prevents the empty `/dashboard` from appearing in history. This is the central role-routing dispatch. Without it, every login redirect would need to know all role routes — coupling `AuthService` to the dashboard structure.

Q123.

How does `AdminProtocolsPageComponent` implement the "check active assignments" before allowing status change? **A:** When the edit modal opens, it fires a background check:

```
this.http.get<any>(`${environment.apiUrl}/site-protocols?protocolId=${p.protocolID}&pageSize=1`)
  .subscribe({ next: r => {
    this.editProtocolHasAssignments = (r.totalCount ?? 0) > 0;
    this.editCheckingAssignments = false;
  }});
```

`pageSize=1` fetches only 1 item but gets `totalCount`. If `totalCount > 0`, the "Terminated" status option is hidden/disabled in the edit form. This mirrors the backend business rule (cannot terminate a protocol with active assignments) but gives the user early feedback in the UI without waiting for a 400 error.

Q124.

How does the audit logs page merge data from two databases? **A:** `AdminAuditLogsPageComponent.loadData()` fires three parallel requests via `forkJoin`: governance DB entries (`/governance-audit-entries`), clinical DB entries (`/clinical-audit-entries`), and login activity (`/audit-logs`). Each has an independent `catchError` — one failing doesn't crash the whole page. The `source` property is tagged on each entry (`'Governance DB'` / `'Clinical DB'`). Both arrays are merged: `[...govItems, ...clinItems].sort((a, b) => new Date(b.changedAt) - new Date(a.changedAt))` — combined and sorted newest-first.

Q125.

What is the `destroy$` pattern used in `NotificationsPageComponent` and how does `takeUntil` prevent a memory leak there? **A:** `NotificationsPageComponent` implements `OnInit` and `OnDestroy`. `private destroy$ = new Subject<void>()`. In `ngOnInit()`: `this.http.get<any>(url).pipe(catchError(...), takeUntil(this.destroy$)).subscribe(...)`. `markRead()` and `deleteNotif()` also use `.pipe(takeUntil(this.destroy$))`. If the user navigates away from the notifications page WHILE an HTTP call is in-flight, `ngOnDestroy` fires `this.destroy$.next()` — all three subscriptions complete and are garbage-collected. Without it, the HTTP callback could fire after the component is destroyed, trying to set `this.notifications` on a dead object.

Q126.

How does the search debounce in `AdminUsersPageComponent` work with pagination? **A:** `onSearch(value: string)` sets `searchTerm` and starts a 400ms timer. The timer calls `loadUsers(1)` — page 1, with the new search term. `loadUsers(page)` builds the query string including `search` if set:

```
const qs = new URLSearchParams({ page: String(page), pageSize: String(this.listPageSize) });
if (this.searchTerm.trim()) qs.set('search', this.searchTerm.trim());
this.http.get<any>(`${environment.apiUrl}/users?${qs}`).subscribe(...)
```

Resetting to page 1 on search ensures you see results from the start, not from wherever you were in the unfiltered list. `ngOnDestroy` cancels the timer if the user navigates away mid-debounce.

Q127.

What is the `DOTNET_ROLE_CLAIM` constant in `AuthService` and why is it needed? **A:** .NET's `ClaimTypes.Role` serializes to `"http://schemas.microsoft.com/ws/2008/06/identity/claims/role"` in the JWT when using the default claim name mapping. But your `JwtTokenService.Generate()` uses `new Claim(ClaimTypes.Role, roleName)` — which your `JwtSecurityTokenHandler.WriteToken()` may serialize as either `"role"` or the full URL. `getRoleFromToken()` checks both:

```
return claims['role'] ?? claims[this.DOTNET_ROLE_CLAIM] ?? null;
```

This ensures the role is extracted regardless of which claim name .NET used during serialization.

Q128.

What is `Object.assign(this.editingAE, payload)` in `InvestigatorAdverseEventsPageComponent.submitEdit()`? **A:** After a successful PUT, instead of reloading all AEs from the server, the component mutates the existing object in-place:

```
Object.assign(this.editingAE, payload);
```

`Object.assign(target, source)` copies all properties from `payload` into `this.editingAE`. Since `this.editingAE` is a reference to the same object in `this.adverseEvents[]`, the array is automatically updated — Angular's change detection picks up the new values and re-renders the row. This is an optimistic update — shows the change immediately without another HTTP round-trip.

Q129.

Why does `AdminUsersPageComponent` prevent editing or deleting users with `isProtected()`? **A:**

```
isSelf(u: any): boolean      { return u?.userID === this.currentUserID; }  
isSuperAdmin(u: any): boolean { return u?.userID === 1; }  
isProtected(u: any): boolean { return this.isSelf(u) || this.isSuperAdmin(u); }
```

`isSelf()` — prevents an admin from accidentally deleting or role-changing their own account (would lock them out).
`isSuperAdmin()` — UserID 1 is the seeded Super Admin; the backend also blocks this (`if (id == 1) return BadRequest`). The frontend guard mirrors the backend protection, giving early feedback rather than waiting for a 400 error.

Q130.

What was the Angular build fix for missing modules? What error appeared and how was it solved? **A:** During development, build errors appeared for missing module imports. `*ngIf` requires `CommonModule`, `[(ngModel)]` requires `FormsModule`, `[formGroup]` requires `ReactiveFormsModule`, `<router-outlet>` requires `RouterModule`. Components declared in `DashboardModule` using reactive forms got `Can't bind to 'formGroup' since it isn't a known property`. Fix: add `ReactiveFormsModule` to `DashboardModule.imports`. The Angular build error identifies the exact directive/component, making it straightforward to trace to the missing module.